

13. Software Quality

By, Prof. Sapna B Patil

Asst. Professor

Dept of CSE, HIT

Contents

1. Introduction
2. The place of software quality in project planning
3. Importance of software quality
4. Defining software quality
5. Software quality models
6. product versus process quality management

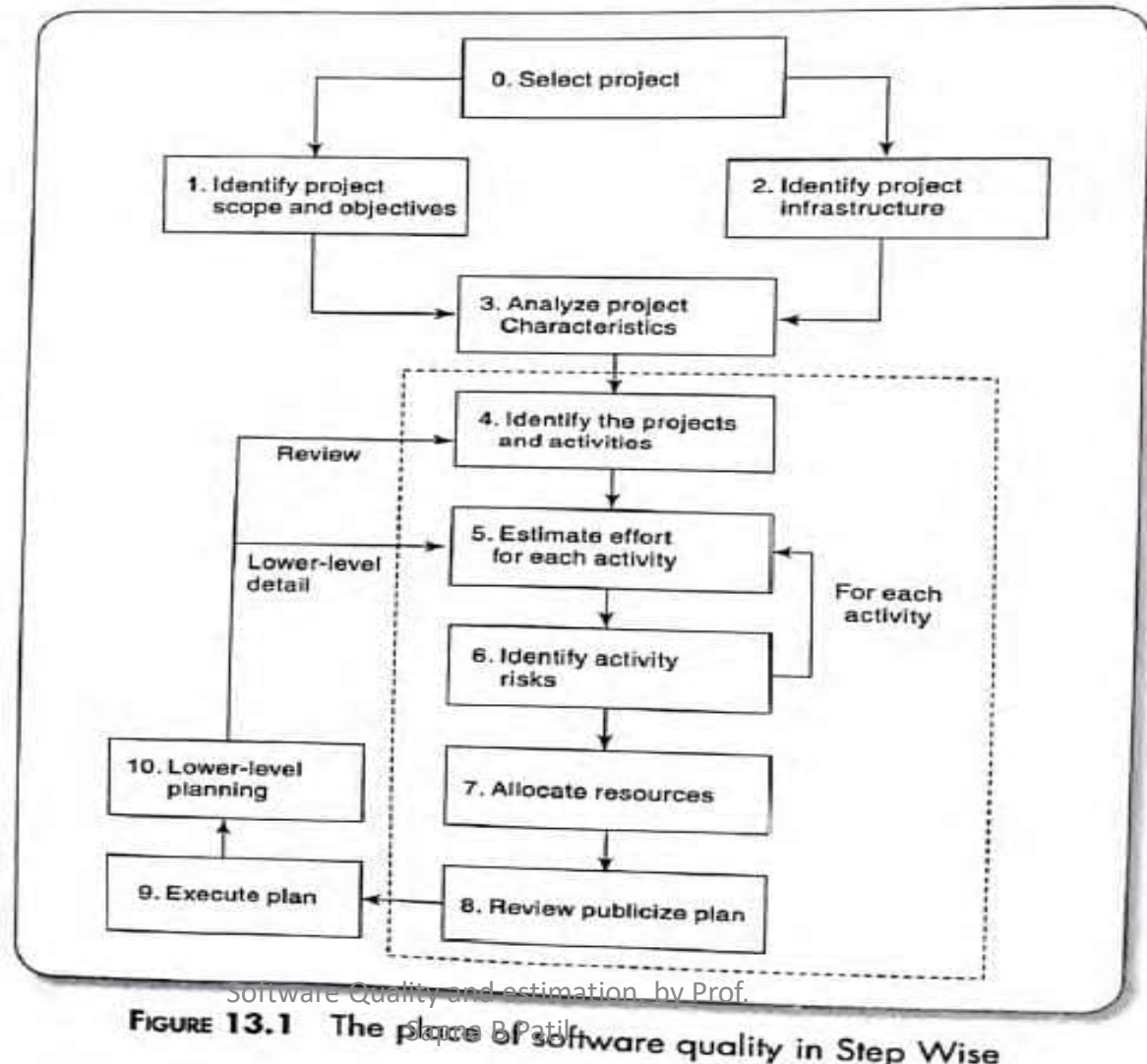
Software Quality

13.1 Introduction

Quality is generally agreed to be a good thing. In a practice the quality of a system can be a vague, undefined, attribute. We therefore need to define precisely what qualities were require of a system. However, this is not enough – we need to objectively weather a system meet out quality requirements and this need measurement.

13.2 The Place of Software Quality in Project Planning

- Quality will be of particular interest at the following points in step wise framework



13.3 Importance of Software Quality

- The special characteristics of software create special demands
 1. Increasing criticality of software:
 2. The intangibility of software :
 3. Accumulating errors during software development:
- For the above reason quality management is an essential part of effective overall project management

13.4 Defining Software Quality

- Defining quality is not enough. If we are to judge whether a system meets our requirements we need to be able to measure its qualities
- A good measure must relate the number of units to the maximum possible.
- Trying to find measures for a particular quality helps to clarify and communicate what quality really is
- The measure may be direct or indirect
- When project managers identify quality measurements they effectively set targets for project team members

13.4 Defining Software Quality

- When there is concern about the need for a specific quality characteristics in a software product then a quality specification with the following minimum details should be drafted
 1. Definition/description : definition of the quality characteristic
 2. Scale: the unit of measurement
 3. Test: the practical test of the extent to which the attribute quality exists
 4. Minimally acceptable: : the worst value which might be acceptable if other characteristics compensated for it , and below which the product would have to be rejected out of hand
 5. Target range: : the range of values within which it is planned the quality measurement value should lie
 6. Now: the value that applies currently

13.4 Defining Software Quality

- There could be several measurements applicable to a quality characteristics, Eg: In the case of reliability, this could be measured in terms of
 1. Availability: The percentage of a particular time interval that a system is usable
 2. Mean Time Between Failure: Total service time /no of failures
 3. Failure on demand: The probability that the system will not be available at the time required
 4. Support activity: The no of fault reports that are generated and processed.
- Associated with reliability is ***maintainability***: which is how quickly a fault once detected can be corrected.
- A key component of this is ***changeability***: which is the ease with which the software can be modified.
- ***Analyzability***: which is the ease with which causes of failure can be identified.

13.5 Software Quality Models

- The need to be able to quantitatively measure the quality of a software is often felt, to verify whether a software meets the quality requirements set for it.
- Unfortunately it is hard to directly measure the quality of a software, However it can be expressed in terms of several attributes of a software that can be directly measured.
- The quality models give a characterization (often hierarchical) of software quality in terms of a set of characteristics of the software.
- We briefly discuss the following models in the upcoming section

Gravin's quality dimensions

- In BOOK- Total quality management
- Gravin defined quality of any products in terms of 8 general attributes of the product
 1. **Performance:** How well it performs the jobs.
 2. **Features:** How well it supports the required features.
 3. **Reliability:** Probability of a product working satisfactorily within a specific period of time.
 4. **Conformance:** Degree to which the product meets the requirements.
 5. **Durability:** Measure of the product life .
 6. **Serviceability:** Speed and effectiveness maintenance
 7. **Aesthetics::** The look and feel of the product.
 8. **Perceived quality** : User's opinion about the product quality

McCall' model

- McCall defined the quality of a software in terms broad parameters **a) its operational characteristics b) how easy is it to fix defects c) how easy it is to port it to different platform**
- Above 3 high level quality attributes are defined based on the following 11 attributes of the software:
 1. Correctness: The extent to which a software product satisfies its specifications.
 2. Reliability: The probability of the software product working satisfactorily over a given duration
 3. Efficiency: The amount of computing resources required to perform the required functions.
 4. Integrity: The extent to which the data of the software product remains valid.
 5. Usability: The effort required to operate the software product.
 6. Maintainability: The ease with which it is possible to locate and fix bugs in the software product
 7. Flexibility: The effort required to adapt the software product to changing requirements.
 8. Testability: The effort required to test a software product to ensure that it performs its intended function.
 9. Portability: The effort required to transfer the software product from one hardware or software system environment to another.
 10. Reusability: The extent to which a software can be reused in other applications.
 11. Interoperability: The effort required to integrate the software with other software. The effort required to integrate the software with other software.

Dromey's Model

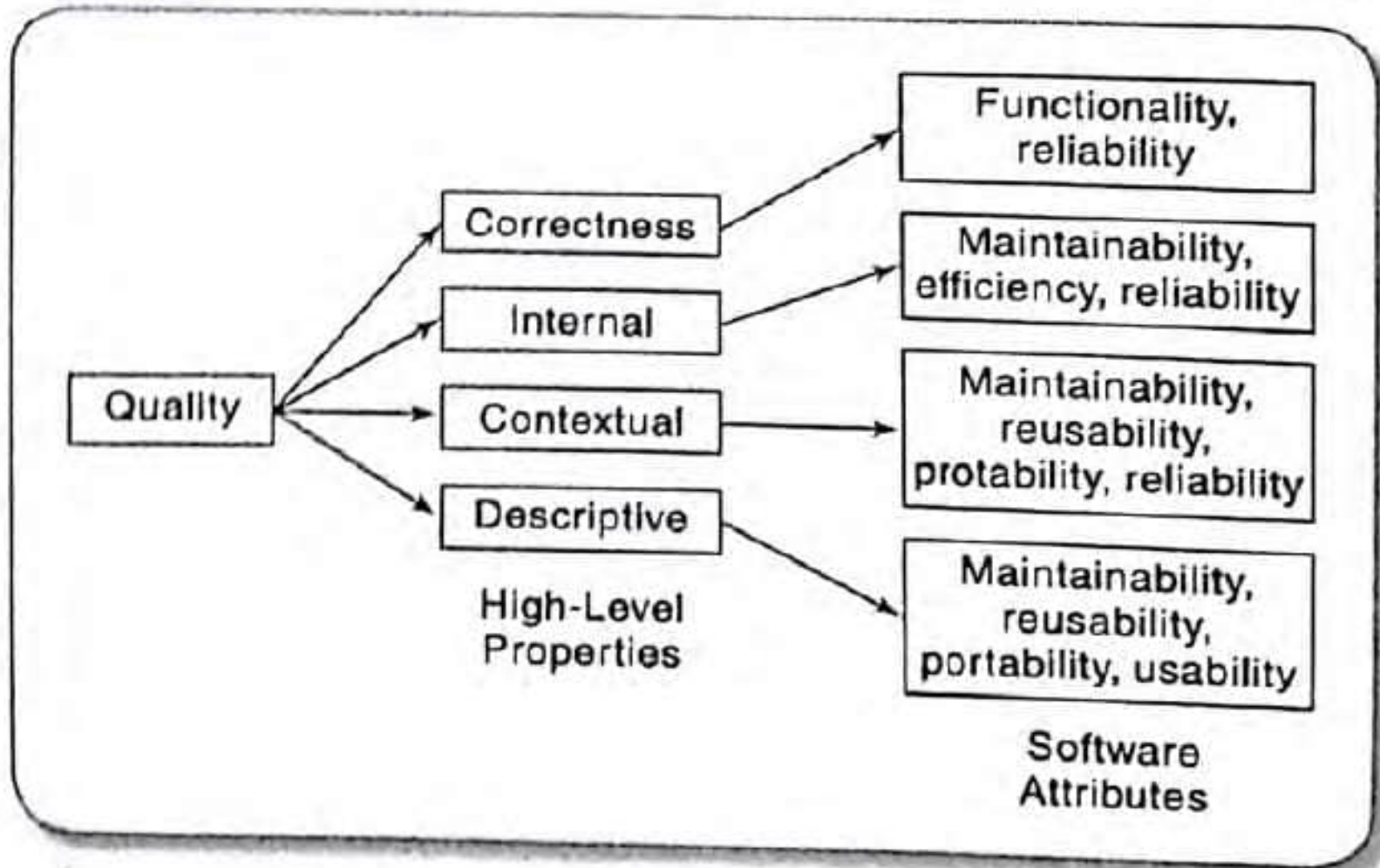


FIGURE 13.2 Dromey's quality model

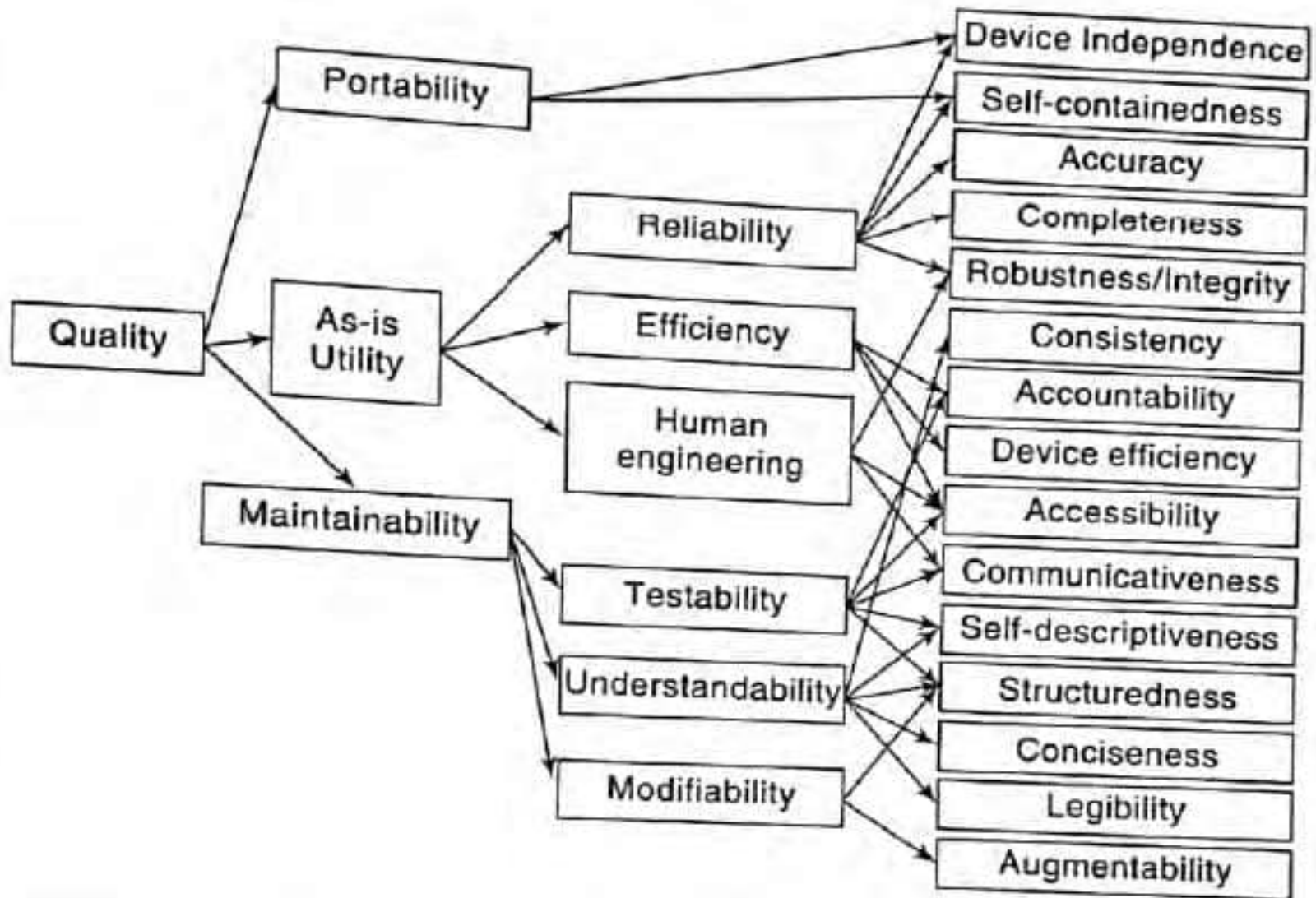
Software Quality and estimation, by Prof.

Sapna B Patil

Boehm's Model

Boehm postulated 3 high level characteristics that are important for the users of the software

1. As-is utility: How well (easily, reliably and efficiently) can it be used?
2. Maintainability: How easy is it to understand, modify and then retest the s/w
3. Portability: How difficult would it be to make the s/w in a changed environment



13.8 Product versus Process Quality Management

- The measurements described above relate to *products*, With a product-based approach to planning and control
- The focus on products is convenient, it is often easier to measure these product qualities in a completed computer application rather than during its development.
- An alternative approach is to scrutinize the quality of *the process* used to develop software product.
- The system development process comprises a number of activities linked so that the output from one activity is the input to the next
- Errors can enter the process at any stage, They can be caused either by defects in a process, as when software developer make mistakes in the logic of their software, or by information not passing clearly and accurately between development stages

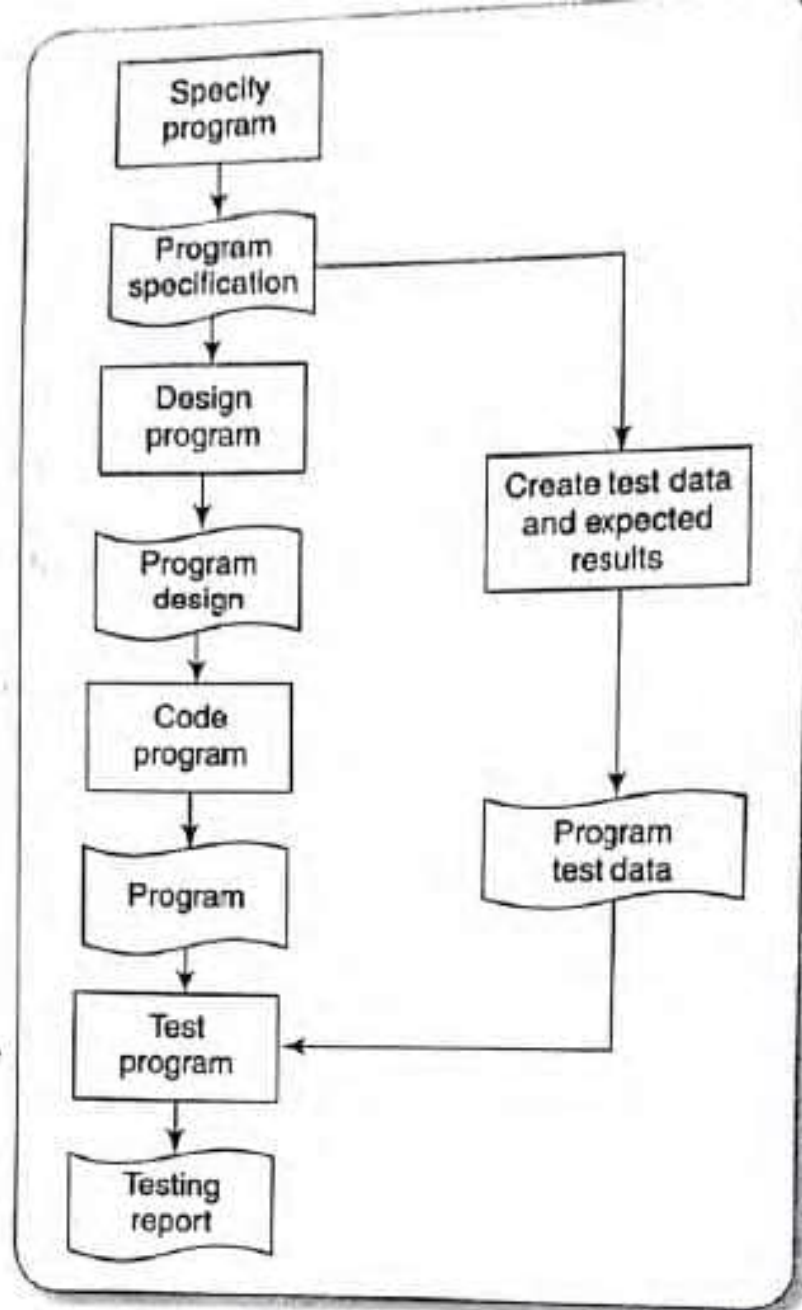


FIGURE 13.4 An example of the sequence of processes and deliverables

Product versus Process Quality Management

Errors that creep in at the early stages are more expensive to correct at late stages for the following reasons

The later the error is found the more rework at more stages of development.

The general tendency is for each successive stage of development to be more detailed and less able to absorb change.

Error should therefore be eradicated by careful examination of the products of each stage before they are passed on to the next. The following process requirement should be specified for each activity.

- 1 Entry requirement. Which have to be in place before an activity can start.
- 2 Implementation requirement: Which define how the process is to be conducted.
- 3 Exit Requirement. Which have to be fulfilled before an activity is deemed to have been completed.

Text data and expected Results

Test
Runs

Run all test

Estimation of Software Projects

26.1 Observation on Estimation

26.6 Decomposition Techniques

26.7 Empirical Estimation Models

Software Project Planning

The overall goal of project planning is to establish a pragmatic strategy for controlling, tracking, and monitoring a complex technical project.

Why?

So the end result gets done on time, with quality!

Project Planning Task Set-I

- Establish project scope
- Determine feasibility
- Analyze risks
 - Risk analysis is considered in detail in Chapter 25.
- Define required resources
 - Determine require human resources
 - Define reusable software resources
 - Identify environmental resources

Project Planning Task Set-II

- Estimate cost and effort
 - Decompose the problem
 - Develop two or more estimates using size, function points, process tasks or use-cases
 - Reconcile the estimates
- Develop a project schedule
 - Scheduling is considered in detail in Chapter 27.
 - Establish a meaningful task set
 - Define a task network
 - Use scheduling tools to develop a timeline chart
 - Define schedule tracking mechanisms

26.1 Observation on Estimation

- Whenever estimates are made, you look into the future and accept some degree of uncertainty as a matter of course.
- Useful techniques for time and effort estimation do exist.
 1. Process and project metrics(quantitative estimates)
 2. Past experience
- We would be ill advised to embark without estimation.
- Estimation of resources, cost, and schedule for a software engineering effort requires
 1. Experience
 2. access to good historical information (metrics)
 3. Courage to commit to quantitative predictions when qualitative information is all that exists.
- Estimation carries inherent risk, and this risk leads to uncertainty.

Continued...

- *Project complexity* has a strong effect on the uncertainty inherent in planning.
- *Project size* is another important factor that can affect the accuracy and efficacy of estimates. As size increases, the interdependency among various elements of the software grows rapidly.
- The degree of *structural uncertainty* also has an effect on estimation risk. (requirements, compartmentalization of functions, hierarchical nature of the information)
- The availability of *historical information* has a strong influence on estimation risk.
- Estimation risk is measured by the degree of uncertainty in the quantitative estimates established for resources, cost, and schedule.
- However, you should not become obsessive about estimation.

26.6 Decomposition Techniques

- Software project estimation is a form of problem solving, and in most cases, the problem to be is too complex to be considered in one piece.
- For this reason, you should decompose the problem.
- decomposition of the problem and decomposition of the process. Estimation uses one or both forms of partitioning.
- But before an estimate can be made, you must understand the scope of the software to be built and generate an estimate of its “size.”

26.6.1 Software Sizing

- The accuracy of a software project estimate is predicated on a number of things:
 1. The degree to which you have properly **estimated the size** of the product to be built;
 2. The ability to translate the size estimate into **human effort, calendar time, and dollars**
 3. The degree to which the **project plan** reflects the abilities of the software team;
 4. The stability of **product requirements** and the **environment** that supports the software engineering effort.
- A project estimate is only as good as the estimate of the size of the work to be accomplished
- In the context of project planning, size refers to a quantifiable outcome of the software project.
- **If a direct approach is taken, size can be measured in lines of code (LOC). If an indirect approach is chosen, size is represented as function points (FP).**

26.6.1 Software Sizing (Conti..)

- Putnam and Myers suggest four different approaches to the sizing problem:
 - “Fuzzy logic” sizing:** To apply this approach, the planner must identify the type of application, establish its magnitude on a qualitative scale, and then refine the magnitude within the original range.
 - Function point sizing:** *The planner develops estimates of the information domain characteristics.*
 - Standard component sizing:** *Software is composed of a number of different “standard components” that are generic to a particular application area. For example, the standard components for an information system are subsystems, modules, screens, reports, interactive programs, batch programs, files, LOC, and object-level instructions.*
 - Change sizing:** *This approach is used when a project encompasses the use of existing software that must be modified in some way as part of a project. The planner estimates the number and type (e.g., reuse, adding code, changing code, deleting code) of modifications that must be accomplished.*

26.6.2 Problem Based Estimation

LOC and FP Estimation

- **lines of code** and **function points** are measures from which productivity metrics can be computed.
- LOC and FP data are used in two ways during software project estimation:
 - (1) as **estimation variables** to “size” each element of the software and
 - (2) as **baseline metrics** collected from past projects and used in conjunction with estimation variables to develop cost and effort projections.

26.6.2 Problem Based Estimation(Conti..)

- LOC and FP estimation are distinct estimation techniques. Yet both have a number of characteristics in common.
 1. Begin with a bounded statement of software scope
 2. From this statement , decompose the statement of scope into problem functions that can each be estimated individually.
 3. Compute LOC or FP (the estimation variable) values for each function.
 4. Derive cost or effort estimates by applying the LOC or FP values to your baseline productivity metrics (e.g., LOC/pm or FP/pm).
 5. Combine function estimates to produce an overall estimate for the entire project.

26.6.2 Problem Based Estimation(Conti..)

- LOC/pm or FP/pm averages should be computed by project domain.
- projects should be grouped by team size, application area, complexity, and other relevant parameters.
- Local domain averages should then be computed.
- The LOC and FP estimation techniques differ in the level of detail required for decomposition and the target of the partitioning
- In LOC, decomposition is absolutely essential and is often taken to considerable levels of detail.
- For FP estimates, each of the information domain characteristics—inputs, outputs, data files, inquiries, and external interfaces—as well as the 14 complexity adjustment values are estimated.
- Regardless of the estimation variable that is used, you should begin by estimating a range of values for each function or information domain value,
- Using historical data or (when all else fails) intuition, estimate an optimistic, most likely, and pessimistic size value for each function or count for each information domain value.

26.6.2 Problem Based Estimation(Conti..)

- A three-point or expected value can then be computed. The *expected value for the estimation variable (size) S can be computed as a weighted average of the optimistic (S_{opt}), most likely (S_m), pessimistic (S_{pess}) estimates. For example*

$$S = \frac{S_{opt} + 4S_m + S_{pess}}{6}$$

- Once the expected value for the estimation variable has been determined, historical LOC or FP productivity data are applied.
- Are the estimates correct?

26.6.3 An Example of LOC-Based Estimation

- I consider a software package to be developed for a computer-aided design application for mechanical components.
- A preliminary statement of software scope can be developed:

The mechanical CAD software will accept two- and three-dimensional geometric data from an engineer. The engineer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of graphics devices. The software will be designed to control and interact with peripheral devices that include a mouse, digitizer, laser printer, and plotter.

- For example, the range of LOC estimates for the 3D geometric analysis function is optimistic, 4600 LOC; most likely, 6900 LOC; and pessimistic, 8600 LOC. Applying Equation 26.1, the expected value for the 3D geometric analysis function is 6800 LOC.

Example: LOC Approach

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	<i>33,200</i>

Average productivity for systems of this type = 620 LOC/pm.

Burdened labor rate =\$8000 per month, the cost per line of code is approximately \$13.

Based on the LOC estimate and the historical productivity data, the total estimated project cost is **\$431,000** and the **estimated effort is 54 person-months**.

26.6.4 An Example of FP-Based Estimation

How FP based approach works?

FIGURE 23.1

Computing function points	Information Domain Value	Count	Weighting factor				
			Simple	Average	Complex		
	External Inputs (EIs)	×	3	4	6	=	
	External Outputs (EOs)	×	4	5	7	=	
	External Inquiries (EQs)	×	3	4	6	=	
	Internal Logical Files (ILFs)	×	7	10	15	=	
	External Interface Files (EIFs)	×	5	7	10	=	
	Count total	→					

$$\text{FP} = \text{count total} \times [0.65 + (0.01 \times \sum Fi)]$$

•where count total is the sum of all FP entries obtained

26.6.4 An Example of FP-Based Estimation

How FP based approach works?(Conti..)

The F_i ($i = 1$ to 14) are value adjustment factors (VAF) based on responses to the following questions [Lon02]:

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

26.6.4 An Example of FP-Based Estimation

FIGURE 26.3

Estimating
information
domain values

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	4	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
<i>Count total</i>						320

26.6.4 An Example of FP-Based Estimation

Factor	Value
Backup and recovery	4
Data communications	2
Distributed processing	0
Performance critical	4
Existing operating environment	3
Online data entry	4
Input transaction over multiple screens	5
Master files updated online	3
Information domain values complex	5
Internal processing complex	5
Code designed for reuse	4
Conversion/installation in design	3
Multiple installations	5
Application designed for change	5
Value adjustment factor	52

$$FP_{\text{estimated}} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)] = 375$$

26.6.4 An Example of FP-Based Estimation

- The organizational average productivity for systems of this type is 6.5 FP/pm.
- Based on a burdened labor rate of \$8000 per month, the cost per FP is approximately \$1230.
- Based on the FP estimate and the historical productivity data,
- the total estimated project cost is \$461,000
- and the estimated effort is 58 person-months.

26.6.5 Process Based Estimation

- The most common technique for estimating a project is to base the estimate on the process that will be used.
- That is, the process is decomposed into a relatively small set of activities, actions and tasks and the effort required to accomplish each is estimated.
- process-based estimation begins with a delineation of software functions obtained from the project scope.
- A series of framework activities must be performed for each function.
- Then we estimate the effort (e.g., person-months) that will be required to accomplish each software process activity for each software function.
- Average labor rates (i.e., cost/unit effort) are then applied to the effort estimated for each process activity
- Costs and effort for each function and framework activity are computed as the last step.

26.6 An example of Process Based Estimation

Process-Based Estimation Example

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function ↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
<i>Totals</i>	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
<i>% effort</i>	1%	1%	1%	8%	45%	10%	36%		

CC = customer communication CE = customer evaluation

Based on an average burdened labor rate of \$8,000 per month, **the total estimated project cost is \$368,000 and the estimated effort is 46 person-months.**

26.6.7 Estimation with Use Cases

- Use cases provide a software team with insight into software scope and requirements.
- Use cases can be used to estimate the project size of a software project.
- Challenges
 1. Use cases are described using many different formats and styles
 2. Use cases represent an external view (the user's view) of the software and can therefore be written at many different levels of abstraction.
 3. Use cases do not address the complexity of the functions and features that are described.
 4. Use cases can describe complex behavior (e.g., interactions) that involve many functions and features.
- Smith [Smi99] suggests that use cases can be used for estimation, but only if they are considered within the context of the “structural hierarchy” that they are used to describe.
- Therefore, before use cases can be used for estimation, the level within the structural hierarchy is established, the average length (in pages) of each use case is determined, the type of software (e.g., real-time, business, engineering/scientific, WebApp, embedded) is defined, and a rough architecture for the system is considered.
- Once these characteristics are established, empirical data may be used to establish the estimated number of LOC or FP per use case (for each level of the hierarchy).
- Historical data are then used to compute the effort required to develop the system.

26.6.7 Estimation with Use Cases(Conti..)

- To illustrate how this computation might be made, consider the following relationship:

$$\text{LOC estimate} = N \times \text{LOCavg} [(S_a/S_h - 1) (P_a/P_h - 1)] \times \text{LOCadjust}$$

where

N = actual number of use cases

LOCavg = historical average LOC per use case for this type of subsystem

LOCadjust = represents an adjustment based on *n percent of LOCavg where n* is defined locally and represents the difference between this project and “average” projects

S_a = actual scenarios per use case

S_h = average scenarios per use case for this type of subsystem

P_a = actual pages per use case

P_h = average pages per use case for this type of subsystem

- Expression (26.2) could be used to develop a rough estimate of the number of LOC based on the actual number of use cases adjusted by the number of scenarios and the page length of the use cases.

26.6.8 An Example of Use Case Based Estimation

Estimation with Use-Cases

	use cases	scenarios	pages	scenarios	pages	LOC	LOC estimate
User interface subsystem	6	10	6	12	5	560	3,366
Engineering subsystem group	10	20	8	16	8	3100	31,233
Infrastructure subsystem group	5	6	5	10	6	1650	7,970
Total LOC estimate							42,568

Using 620 LOC/pm as the average productivity for systems of this type and a burdened labor rate of \$8000 per month, the cost per line of code is approximately \$13. Based on the use-case estimate and the historical productivity data, **the total estimated project cost is \$553,397 and the estimated effort is**

26.6.9 Reconciling Estimates

- Multiple estimates that must be reconciled to produce a single estimate of effort, project duration, or cost.
- The total estimated effort for the CAD software ranges from a low of 46 person months (derived using a process-based estimation approach) to a high of 68 person months (derived with use-case estimation). The average estimate (using all four approaches) is 56 person-months. The variation from the average estimate is approximately 18 percent on the low side and 21 percent on the high side.
- What happens when agreement between estimates is poor?
- Widely divergent estimates can often be traced to one of two causes
 - (1) the scope of the project is not adequately understood or has been misinterpreted by the planner, or
 - (2) Productivity data used for problem-based estimation techniques is inappropriate for the application, obsolete (in that it no longer accurately reflects the software engineering organization), or has been misapplied.

26.7 Empirical Estimation Models

- An estimation model for computer software uses empirically derived formulas to predict effort as a function of LOC or FP.
- instead of using the tables , the resultant values for LOC or FP are plugged into the estimation model.
- The empirical data that support most estimation models are derived from a limited sample of projects.
- No estimation model is appropriate for all classes of software and in all development environments.
- The model should be tested by applying data collected from completed projects, plugging the data into the model, and then comparing actual to predicted results.
- If agreement is poor, the model must be tuned and retested before it can be used.

26.7.1 The Structure of Estimation Models

- A typical estimation model is derived using regression analysis on data collected from past software projects.
- The overall structure of such models takes the form

$$E = A + B \times (e_v)^C$$

- where A , B , and C are empirically derived constants
- E is effort in person-months,
- ev is the estimation variable (either LOC or FP).
- the majority of estimation models have some form of **project adjustment component** that enables E to be adjusted by other project characteristics (e.g., problem complexity, staff experience, development environment).

26.7.1 The Structure of Estimation Models(Conti..)

- Among the many LOC-oriented estimation models proposed in the literature are

$$E = 5.2 \times (\text{KLOC})^{0.91}$$

$$E = 5.5 + 0.73 \times (\text{KLOC})^{1.16}$$

$$E = 3.2 \times (\text{KLOC})^{1.05}$$

$$E = 5.288 \times (\text{KLOC})^{1.047}$$

Walston-Felix model

Bailey-Basili model

Boehm simple model

Doty model for KLOC > 9

- FP-oriented models have also been proposed. These include

$$E = -91.4 + 0.355 \text{ FP}$$

$$E = -37 + 0.96 \text{ FP}$$

$$E = -12.88 + 0.405 \text{ FP}$$

Albrecht and Gaffney model

Kemerer model

Small project regression model

26.7.2 The COCOMO II Model

- Barry Boehm [Boe81] introduced a hierarchy of software estimation models bearing the name COCOMO, for *CO*nstructive *CO*st *MO*del.
- The original COCOMO model became one of the most widely used and discussed software cost estimation models in the industry
- It has evolved into a more comprehensive estimation model, called COCOMOII
- COCOMO II is actually a hierarchy of estimation models that address the following areas:
 - Application Composition Model
 - Early design stage model
 - Post architecture stage model
- the COCOMO II models require sizing information.
- Three different sizing options are available as part of the model hierarchy:
 1. Object points
 2. Function points
 3. Lines of source code.

26.7.2 The COCOMO II Model(Conti..)

STEPS FOR EFFORT ESTIMATION

1. **Assess Object Points:** Like function points, the *object point is an indirect software measure that is computed* using counts of the number of (1) **screens** (at the user interface), (2) **reports** and (3) **3GL components** likely to be required to build the application.
2. **Classify Complexity levels of each Object:** Each object instance (e.g., a screen or report) is classified into one of three complexity levels (i.e., simple, medium, or difficult) using criteria suggested by Boehm.

FIGURE 26.6

Complexity weighting for object types.

Source: [Boe96].

Object type	Complexity weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL component			10

STEPS FOR EFFORT ESTIMATION(Conti,,)

3. Assign Complexity weight to each object: Once complexity is determined, the number of screens, reports, and components are weighted according to the table illustrated in Figure 26.6.
4. Determine Object Points: The object point count is then determined by multiplying the original number of object instances by the weighting factor in the figure and summing to obtain a total object point count.
5. Compute New Object point: When component-based development or general software reuse is to be applied, the percent of reuse (%reuse) is estimated and the object point count is adjusted:

$$\text{New Object Point (NOP) = (Object points) x [(100 - \%reuse)/100]}$$

STEPS FOR EFFORT ESTIMATION(Conti..)

- 6. Calculate Productivity rate(PROD):** To derive an estimate of effort based on the computed NOP value, a “productivity rate” must be derived. Figure 26.7 presents the productivity rate.

FIGURE 26.7 Productivity rate for object points.
Source: [Boe96].

Developer's experience/capability	Very low	Low	Nominal	High	Very high
Environment maturity/capability	Very low	Low	Nominal	High	Very high
PROD	4	7	13	25	50

$$\text{PROD} = \frac{\text{NOP}}{\text{person-month}}$$

- 7. Compute Effort in Person – Month:** Once the productivity rate has been determined, an estimate of project effort is computed using

$$\text{Estimated effort} = \frac{\text{NOP}}{\text{PROD}}$$

26.7.3 The Software Equation

- The *software equation* [Put92] is a dynamic multivariable model that assumes a specific distribution of effort over the life of a software development project.
- derived from productivity data collected for over 4000 contemporary software projects.

$$E = \frac{\text{LOC} \times B^{0.333}}{P^3} \times \frac{1}{t^4} \quad (26.4)$$

where

E = effort in person-months or person-years

t = project duration in months or years

B = "special skills factor"¹³

P = "productivity parameter" that reflects: overall process maturity and management practices, the extent to which good software engineering practices are used, the level of programming languages used, the state of the software environment, the skills and experience of the software team, and the complexity of the application

2000, 10,000 &
28000



26.7.3 The Software Equation(Conti..)

- You should note that the software equation has two independent parameters:
 - (1) an estimate of size (in LOC) and
 - (2) an indication of project duration in calendar months or years.
- To simplify the estimation process ,Putnam and Myers [Put92] suggest a set of equations derived from the software equation.
- Minimum development time is defined as

$$t_{\min} = 8.14 \frac{\text{LOC}}{p^{0.43}} \text{ in months for } t_{\min} > 6 \text{ months} \quad (26.5a)$$

$$E = 180 Bt^3 \text{ in person-months for } E \geq 20 \text{ person-months} \quad (26.5b)$$

26.7.3 The Software Equation(Conti..)

- Using Equation (26.5) with P 12,000 (*the recommended value for scientific software*) for the CAD software discussed earlier in this chapter.

$$t_{\min} = 8.14 \times \frac{33,200}{12,000^{0.43}} = 12.6 \text{ calendar months}$$

$$E = 180 \times 0.28 \times (1.05)^3 = 58 \text{ person-months}$$

- The results of the software equation correspond favorably with the estimates developed in Section 26.6. Like the COCOMO model noted in Section 26.7.2, the software equation continues to evolve.